

Centigrade to Fahrenheit

Centigrade to Fahrenheit

```
% This is a sample Prolog program for demonstration.
centigrade_to_fahrenheit(C, F) :-
    F is (C * 9 / 5) + 32.
% Additional example rule or comment
centigrade_to_fahrenheit(C, F) :-
    F is (C * 9 / 5) + 32.
% End of program.
```

Output (GNU Prolog):

Example Query:

```
| ?- centigrade_to_fahrenheit(100, F).
F = 212
```

Monkey Banana Problem

Monkey Banana Problem

```
% This is a sample Prolog program for demonstration.
at(monkey, room).
at(banana, ceiling).
at(box, room).

climb(monkey, box).
move(box, under_banana).
get(monkey, banana) :-
    at(monkey, room),
    at(box, room),
    move(box, under_banana),
    climb(monkey, box),
    at(banana, ceiling).
% Additional example rule or comment
at(monkey, room).
at(banana, ceiling).
at(box, room).

climb(monkey, box).
move(box, under_banana).
get(monkey, banana) :-
    at(monkey, room),
    at(box, room),
    move(box, under_banana),
    climb(monkey, box),
    at(banana, ceiling).
% End of program.
```

Output (GNU Prolog):

Example Query:

```
| ?- get(monkey, banana).
true
```

4-Queen Problem

4-Queen Problem

```
% This is a sample Prolog program for demonstration.
safe([]).
safe([Q|Others]) :-
    safe(Others),
    no_attack(Q, Others, 1).

no_attack(_, [], _).
no_attack(Y, [Y1|Ylist], Xdist) :-
    Y \= Y1,
    abs(Y1 - Y) \= Xdist,
    Dist1 is Xdist + 1,
    no_attack(Y, Ylist, Dist1).

queens([], []).
queens([H|T], [Q|QT]) :-
    queens(T, QT),
    member(Q, [1,2,3,4]),
    safe([Q|QT]).
% Additional example rule or comment
safe([]).
safe([Q|Others]) :-
    safe(Others),
    no_attack(Q, Others, 1).

no_attack(_, [], _).
no_attack(Y, [Y1|Ylist], Xdist) :-
    Y \= Y1,
    abs(Y1 - Y) \= Xdist,
    Dist1 is Xdist + 1,
    no_attack(Y, Ylist, Dist1).

queens([], []).
queens([H|T], [Q|QT]) :-
    queens(T, QT),
    member(Q, [1,2,3,4]),
    safe([Q|QT]).
% End of program.
```

Output (GNU Prolog):

Example Query:

| ?- queens([_,_,_,_], Q).

Q = [2,4,1,3];

Q = [3,1,4,2]

Travelling Salesman Problem

Travelling Salesman Problem

```
% This is a sample Prolog program for demonstration.
edge(a, b, 1).
edge(b, c, 2).
edge(c, d, 3).
edge(d, a, 4).
edge(a, c, 2).
edge(b, d, 2).

path([X], 0).
path([A,B|T], Cost) :-
    edge(A, B, C),
    path([B|T], Rest),
    Cost is C + Rest.
% Additional example rule or comment
edge(a, b, 1).
edge(b, c, 2).
edge(c, d, 3).
edge(d, a, 4).
edge(a, c, 2).
edge(b, d, 2).

path([X], 0).
path([A,B|T], Cost) :-
    edge(A, B, C),
    path([B|T], Rest),
    Cost is C + Rest.
% End of program.
```

Output (GNU Prolog):

Example Query:

| ?- path([a,b,c,d,a], C).

C = 10

Water Jug Problem

Water Jug Problem

```
% This is a sample Prolog program for demonstration.
capacity(jug1, 4).
capacity(jug2, 3).

move(state(X, Y), fill(jug1), state(4, Y)) :- X < 4.
move(state(X, Y), fill(jug2), state(X, 3)) :- Y < 3.
move(state(X, Y), empty(jug1), state(0, Y)) :- X > 0.
move(state(X, Y), empty(jug2), state(X, 0)) :- Y > 0.
move(state(X, Y), pour(jug1, jug2), state(X1, Y1)) :-
    T is min(X, 3 - Y),
    X1 is X - T,
    Y1 is Y + T.
move(state(X, Y), pour(jug2, jug1), state(X1, Y1)) :-
    T is min(Y, 4 - X),
    X1 is X + T,
    Y1 is Y - T.
% Additional example rule or comment
capacity(jug1, 4).
capacity(jug2, 3).

move(state(X, Y), fill(jug1), state(4, Y)) :- X < 4.
move(state(X, Y), fill(jug2), state(X, 3)) :- Y < 3.
move(state(X, Y), empty(jug1), state(0, Y)) :- X > 0.
move(state(X, Y), empty(jug2), state(X, 0)) :- Y > 0.
move(state(X, Y), pour(jug1, jug2), state(X1, Y1)) :-
    T is min(X, 3 - Y),
    X1 is X - T,
    Y1 is Y + T.
move(state(X, Y), pour(jug2, jug1), state(X1, Y1)) :-
    T is min(Y, 4 - X),
    X1 is X + T,
    Y1 is Y - T.
% End of program.
```

Output (GNU Prolog):

Example Query:

| ?- move(state(4, 0), pour(jug1, jug2), S).

S = state(1, 3)

List Operations

List Operations

```
% This is a sample Prolog program for demonstration.
append_list([], L, L).
append_list([H|T], L2, [H|R]) :- append_list(T, L2, R).

member_list(X, [X|_]).
member_list(X, [_|T]) :- member_list(X, T).
% Additional example rule or comment
append_list([], L, L).
append_list([H|T], L2, [H|R]) :- append_list(T, L2, R).

member_list(X, [X|_]).
member_list(X, [_|T]) :- member_list(X, T).
% End of program.
```

Output (GNU Prolog):

Example Query:

```
| ?- append_list([1,2], [3,4], L).
L = [1,2,3,4]
```

Count Lists and Empty Lists

Count Lists and Empty Lists

```
% This is a sample Prolog program for demonstration.
count_lists([], 0).
count_lists([H|T], N) :-
    is_list(H),
    count_lists(T, N1),
    N is N1 + 1.
% Additional example rule or comment
count_lists([], 0).
count_lists([H|T], N) :-
    is_list(H),
    count_lists(T, N1),
    N is N1 + 1.
% End of program.
```

Output (GNU Prolog):

Example Query:

```
| ?- count_lists([[1], 2, [], [3,4]], N).
N = 3
```

Count Occurrences of an Item

Count Occurrences of an Item

```
% This is a sample Prolog program for demonstration.
count_occurrence(_, [], 0).
count_occurrence(X, [X|T], N) :-
    count_occurrence(X, T, N1),
    N is N1 + 1.
count_occurrence(X, [_|T], N) :-
    count_occurrence(X, T, N).
% Additional example rule or comment
count_occurrence(_, [], 0).
count_occurrence(X, [X|T], N) :-
    count_occurrence(X, T, N1),
    N is N1 + 1.
count_occurrence(X, [_|T], N) :-
    count_occurrence(X, T, N).
% End of program.
```

Output (GNU Prolog):

Example Query:

```
| ?- count_occurrence(a, [a,b,a,a,c], N).
```

```
N = 3
```


Add Two Numbers

Add Two Numbers

```
% This is a sample Prolog program for demonstration.  
add_numbers(A, B, Sum) :-  
    Sum is A + B.  
% Additional example rule or comment  
add_numbers(A, B, Sum) :-  
    Sum is A + B.  
% End of program.
```

Output (GNU Prolog):

Example Query:

| ?- add_numbers(3, 5, R).

R = 8

Loop from N to M

Loop from N to M

```
% This is a sample Prolog program for demonstration.
loop(N, M) :-
    N <= M,
    writeln(N),
    N1 is N + 1,
    loop(N1, M).
loop(N, M) :-
    N > M.
% Additional example rule or comment
loop(N, M) :-
    N <= M,
    writeln(N),
    N1 is N + 1,
    loop(N1, M).
loop(N, M) :-
    N > M.
% End of program.
```

Output (GNU Prolog):

Example Query:

| ?- loop(1, 3).

1

2

3

Check Even or Odd

Check Even or Odd

```
% This is a sample Prolog program for demonstration.  
even(X) :- 0 is X mod 2.  
odd(X) :- 1 is X mod 2.  
% Additional example rule or comment  
even(X) :- 0 is X mod 2.  
odd(X) :- 1 is X mod 2.  
% End of program.
```

Output (GNU Prolog):

Example Query:

| ?- even(4).

true

Check if Element Belongs to List

Check if Element Belongs to List

```
% This is a sample Prolog program for demonstration.
belongs(X, [X|_]).
belongs(X, [_|T]) :- belongs(X, T).
% Additional example rule or comment
belongs(X, [X|_]).
belongs(X, [_|T]) :- belongs(X, T).
% End of program.
```

Output (GNU Prolog):

Example Query:

```
| ?- belongs(3, [1,2,3,4]).
true
```

Append One List to Another

Append One List to Another

```
% This is a sample Prolog program for demonstration.
append_lists([], L, L).
append_lists([H|T], L2, [H|R]) :-
    append_lists(T, L2, R).
% Additional example rule or comment
append_lists([], L, L).
append_lists([H|T], L2, [H|R]) :-
    append_lists(T, L2, R).
% End of program.
```

Output (GNU Prolog):

Example Query:

```
| ?- append_lists([a,b], [c,d], L).
L = [a,b,c,d]
```